

CPSC 213: Introduction to Computer Systems

Unit 1c: Instance Variables and Dynamic Allocation

Jordon Johnson

Adapted from slides by Mike Feeley, Jonatan Schroeder, and Robert Xiao

Overview

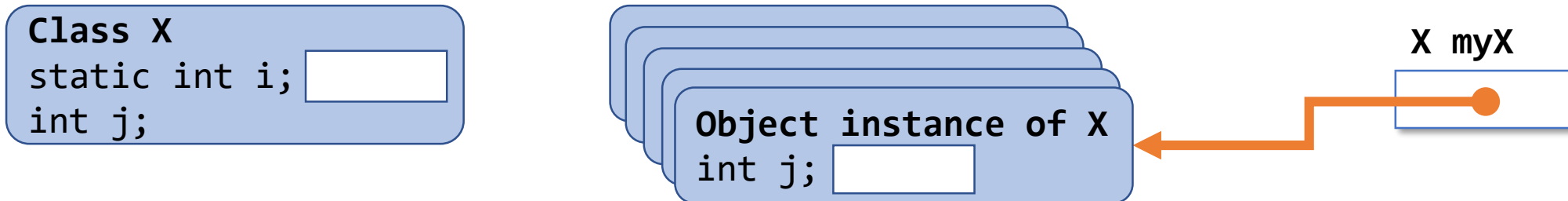
- Reading
 - Companion: 2.4.4 – 2.5
 - Textbook: 3.9.1, 9.9, 3.10 (reference)
- Learning Objectives
 - read and write C code that includes structs
 - compare Java classes/objects with C structs
 - explain the difference between static and non-static variables in Java
 - explain the usage of ISAs' base-plus-offset and indexed addressing modes
 - distinguish static and dynamic computation for access to members of a static struct variable in C
 - distinguish static and dynamic computation for access to members of a non-static struct variable in C
 - read and write assembly code that uses base-plus-offset instructions
 - compare Java dynamic allocation the C's explicit-delete approach by identifying relative strengths and weaknesses
 - identify and correct dangling-pointer and memory leak bugs in C caused by improper use of free()
 - write C code that uses techniques that avoid dynamic allocation as a way to minimize memory-allocation bugs
 - write C code that uses reference counting as a way to minimize memory-allocation bugs
 - explain why Java code does not have dangling-reference errors but can have memory-leak errors

Recap: Variable Types So Far

- Static variables
 - the compiler allocates them: memory address is a constant
- Scalar variable stores a single value
- Array stores multiple values of the same type named by variable and index
 - data can be allocated either statically or dynamically
 - array variable can be the array (static) or store a pointer to it (dynamic)
 - index is generally considered dynamic

Instance Variables

- Variables that are an instance of a **class** or **struct**
 - Created dynamically (usually)
 - Many instances of the same class/struct can co-exist
- **Java** vs **C**
 - Java: **objects** are instances of non-static variables of a **class**
 - C: **structs** are named variable groups, or one of their instances
- Accessing an instance variable
 - requires a reference/pointer to a particular object/struct
 - then variable name chooses a variable in that object/struct



Structs in C

- A struct is a **collection** of variables of arbitrary type
 - **allocated** and **accessed together**
- Declaration
 - similar to declaring a Java class without methods
 - name is “struct” plus name provided by programmer
 - **static:** `struct D d0;`
 - **dynamic:** `struct D* d1;`
- Access:
 - **static:** `d0.e = d0.f;`
 - **dynamic:** `d1->e = d1->f;`

```
struct D {  
    int e;  
    int f;  
}
```

≈

```
class D {  
    public int e;  
    public int f;  
}
```

Struct Allocation

- **Static** structs are allocated by the compiler

Static Memory Layout

```
struct D d0;
```

```
0x1000: value of d0.e  
0x1004: value of d0.f
```

```
struct D {  
    int e;  
    int f;  
}
```

- **Dynamic** structs are allocated at runtime
 - the struct pointer may be **static** or **dynamic**
 - the struct itself is allocated with a call to **malloc**

Static Memory Layout

```
struct D* d1;
```

```
0x1000: value of d1  
        (address of instance)
```

Struct Allocation

- Runtime allocation of **dynamic** struct

```
void foo() {  
    d1 = malloc(sizeof(struct D));  
}
```

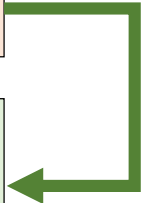
- Example: assume **malloc** returned 0x2000

```
struct D {  
    int e;  
    int f;  
}
```

Static Memory Layout

0x1000: 0x2000 # d1

0x2000: value of d1->e
0x2004: value of d1->f



Struct Access in Assembly

- Struct members can be accessed using **offset**
 - Offset to each variable from base of struct is **static**
- As before, **static** and **dynamic** differ by **extra memory access**

```
struct D {  
    int e;  
    int f;  
}
```

```
d0.e = d0.f;
```

```
d1->e = d1->f;
```

Semantics

```
r[0]    ← 0x1000  
r[1]    ← m[r[0]+4]  
m[r[0]] ← r[1]
```

Assembly

```
ld $0x1000, r0  
ld 4(r0), r1  
st r1, (r0)
```

Semantics

```
r[2]    ← 0x1000  
r[0]    ← m[r[2]]  
r[1]    ← m[r[0]+4]  
m[r[0]] ← r[1]
```

Assembly

```
ld $0x1000, r2  
ld (r2), r0  
ld 4(r0), r1  
st r1, (r0)
```


Recap: Offsets in ISA Specification

- Machine format for base+offset:
 - Offset in our case will always be a multiple of 4
 - What does this mean for the binary representation of our offset?
 - Note that we only have 4 bits to store offset
 - So, we will store $p = o/4$ in the instruction (offset divided by 4)

Name	Semantics	Assembly	Machine
load immediate	$r[d] \leftarrow v$	ld \$v, rd	0d-- vvvvvvvv
load base+offset	$r[d] \leftarrow m[r[s]+o]$	ld o(rs), rd	1psd
load indexed	$r[d] \leftarrow m[r[s]+4*r[i]]$	ld (rs,ri,4), rd	2sid
store base+offset	$m[r[d]+o] \leftarrow r[s]$	st rs, o(rd)	3spd
store indexed	$m[r[d]+4*r[i]] \leftarrow r[s]$	st rs, (rd,ri,4)	4sdi

Recap/Update: Memory Accesses

- Scalars

- access memory at **address** in register

```
ld (r1), r0
```

- Arrays

- **base address** in register
- dynamic **index** in register
- access memory at base plus index times element-size

```
ld (r1,r2,4), r0
```

- Struct members

- **base address** in register (start of struct)
- static (constant) **offset** (bytes from start of struct)
- access memory at base plus static offset
- equivalent to access to array element with **static index**

```
ld 8(r1), r0
```

Given

```
struct X {  
    int i;  
    int j;  
}
```

```
struct X s; // s is a global variable
```

```
ld $s, r0    #r0 = address of s
```

Which of the following loads **s.j** into **r1**?

A `ld $4, r1`

B `ld 4(r0), r1`

C `ld (r0), r1
add 4, r1`

D `ld (r0), r0
add 4, r0
ld (r0), r1`

E none of these

Class activity: offset approaches

array pointer with dynamic index

```
int *b, i;
```

```
b[i]
```

using `ld (rx, ry, 4), r0`

```
ld $b, r0          # r0 = &b
ld (r0), r0         # r0 = b
ld $i, r1           # r1 = &i
ld (r1), r1         # r1 = i
ld (r0, r1, 4), r0 # r0 = b[i]
```

using `ld x(ry), r0`

soon™

struct pointer

```
struct S {
    int i;
    int j;
} *s;
```

```
s->j
```

using `ld x(ry), r0`

```
ld $s, r0          # r0 = &s
ld (r0), r0         # r0 = s
ld 4(r0), r0       # r0 = s->j
```

using `ld (rx, ry, 4), r0`

using ld (rx, ry, 4), r0

```
ld $b, r0          # r0 = &b
ld (r0), r0         # r0 = b
ld $i, r1          # r1 = &i
ld (r1), r1         # r1 = i
ld (r0, r1, 4), r0 # r0 = b[i]
```

using ld x(ry), r0

A ld i(r0), r0

B ld r1(r0), r0

C shl \$2, r1
ld r1(r0), r0

array pointer with dynamic index

```
int *b, i;
```

```
b[i]
```

D shl \$2, r1
add r0, r1
ld (r1), r0

E add r0, r1
ld (r1), r0

Variations in Struct Declaration

- Struct variables can be declared **inside other structs**

```
struct D {  
    int e;  
    int f;  
}
```

```
struct X {  
    int i;  
    struct D d;  
    int j;  
}
```

```
struct Y {  
    int i;  
    struct D* d;  
    int j;  
}
```

```
struct Z {  
    int i;  
    struct Z* z;  
    int j;  
}
```

- Struct members can be **arrays** or **pointers**

```
struct W {  
    int i;  
    int a[10];  
    int j;  
}
```

```
struct W {  
    int i;  
    int* a;  
    int j;  
}
```

What is the offset of member **j** in **struct X** below?

```
struct D {  
    int e;  
    int f;  
}
```

```
struct X {  
    int      i;  
    struct D d;  
    int      j;  
}
```

Struct Size and Alignment

- Alignment rules apply inside a struct
 - Each instance variable will be aligned according to its **type size**
 - Structs are usually aligned according to their **largest member variable type**
- Structs can mix types (**padding** might be added in or after the members)
 - What are the **sizes** of the structs below? **sizeof(struct X)**
 - What are the **offsets** of their members? **offsetof(struct X, i)**

```
struct X {  
    char a;  
    char b;  
    int i;  
}
```

```
struct Y {  
    char a;  
    int i;  
    char b;  
}
```


What is the offset of member **j** in **struct X** below?

Assume 8-byte addresses.

```
struct D {  
    int e;  
    int f;  
}
```

```
struct X {  
    int i;  
    struct D *d;  
    int j;  
}
```

Arrays and Structs

- Array in struct

```
struct S {  
    int i;  
    int j[10];  
}
```

- Array of structs

```
struct S a[42];
```

- Array of pointers to structs

```
struct S* b[42];
```

```
a[3].i = b[7]->j[4];
```

What is `int* e[42];` ?

What is the offset of member **j** in **struct X** below?

```
struct D {  
    int e;  
    int f;  
}
```

```
struct X {  
    int      i;  
    struct D d[10];  
    int      j;  
}
```

What is the offset of member **j** in **struct X** below?

```
struct Y {  
    char  c;  
    short s;  
}
```

```
struct X {  
    int      i;  
    struct Y y[10];  
    int      j;  
}
```

Given

```
struct A {  
    char a;  
    int *b;  
    int *p;  
}
```

```
struct X {  
    char    c;  
    struct A a;  
    int     j;  
}
```

```
struct X s;  // global
```

How many memory reads to load `s.a.b[2]` into `r1`?

- A. 1
- B. 2
- C. 3
- D. 4
- E. not enough information

Dynamic Allocation

Dynamic Allocation in C and Java

- Programs can allocate memory dynamically
 - allocation reserves a range of memory for a purpose
- In **Java**, instances of classes are allocated by the **new** statement
- In **C**, byte ranges are allocated by call to **malloc** procedure
 - these bytes can be used for **any type** that can fit in them

Dynamic Allocation in C

- Memory allocation

```
void* malloc(unsigned long n);
```

- `n` is the number of bytes to allocate
- returning type is `void*`
 - pointer to anything (no specific type assigned)
 - can be cast to/from any other pointer type
 - cannot be dereferenced directly
- Use `sizeof` to determine number of bytes to allocate

```
struct Foo *f = malloc(sizeof(struct Foo));
```

 - `statically` computes number of bytes in type or variable
 - **Caution:** `sizeof(pointer)` gives the size of a **pointer**, not what it **points to**

Memory Deallocation

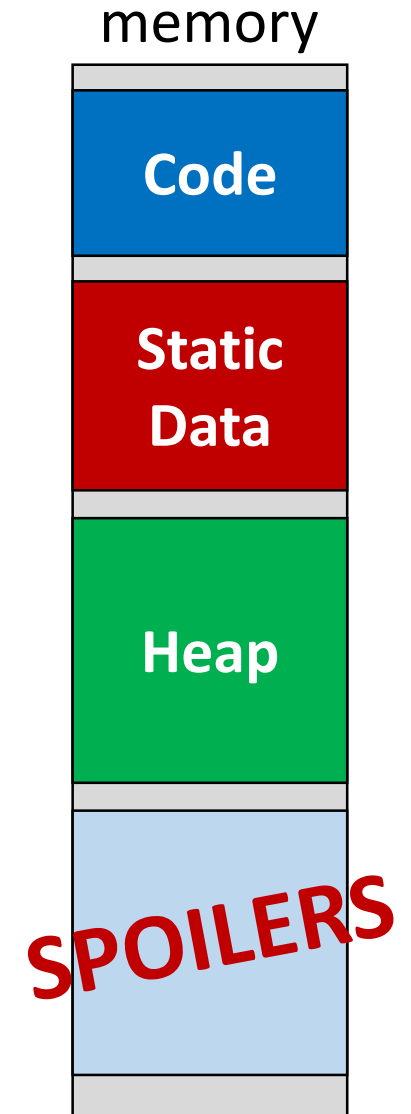
- Wise management of memory requires **deallocation**
 - Memory is a finite resource
 - Deallocation **frees** previously allocated memory for re-use
- In **Java**:
 - **Garbage collection**: memory is deallocated when no longer in use
 - Requires keeping track of every reference to an object
 - Java does it on its own
- In **C**:
 - Dynamic memory must be deallocated **explicitly** by calling **free**
 - Memory is deallocated **immediately, no checks** if it's still in use

Memory Heap

- The **heap** is a large section of memory from which **malloc** allocates objects
 - **malloc** finds unused space in the heap, marks as used and returns it
 - **free** marks space as unused
 - heap may be increased if necessary
- **Java**: all objects are stored in the heap

What malloc and free do

- **Organization of memory**
 - statically allocated: code, static data
 - the rest is unused or dynamically allocated
- **malloc/free**
 - implemented in library
 - manage a chunk of memory called the Heap
 - keep track of what's allocated or free
- **malloc**
 - find a free chunk of memory of appropriate size
 - mark it allocated
 - return pointer to it (keeping track of its size)
- **free**
 - use pointer to determine allocated size
 - mark referenced memory as free



Issues with Explicit Deallocation

- What `free(x)` does
 - deallocates chunk of memory starting at address `x`
 - this memory can be reused by subsequent call to `malloc`
- What `free(x)` does **not** do
 - it does **not** change what `x` points to
 - it does **not** change any other pointers pointing to that memory
 - it does **not** change/"erase" the data in the freed memory

Explicit Deallocation Example

What bad things can happen below?

```
struct Buffer *create() {  
    struct Buffer *buf = malloc(sizeof(struct Buffer));  
    ...  
    return buf;  
}  
void destroy(struct Buffer *buf) {  
    ...  
    free(buf);  
}
```

Dangling Pointers

- A **dangling pointer** is a pointer to an object that is not allocated for the purpose it's been called for
 - often caused by **use-after-free**
 - could point to **unallocated memory**
 - if malloc has been called since last free, could point to **another object**
 - also happens with **incorrect pointer manipulation**
- Why is this a problem?
 - program thinks it's writing to one object, but may be writing to another
 - program may be writing to object of another type

Memory Leaks

- A **memory leak** is a dynamic memory object with **no pointers** pointing to it
 - Usually happens if a program doesn't **free object properly**
 - May also happen if a pointer to valid object is **changed to another value**
- Why is this a problem?
 - If object had useful information, it **can't be accessed anymore**
 - If object is large (or if many memory leaks happen in sequence), program will use **too much memory**

Avoiding Memory Problems in C

- Understand the problem

- where is the memory allocated?
- where is the memory freed?

- Avoid the program cases

- if possible, restrict dynamic allocation/free to **single procedure**
- if possible, don't write procedures that return pointers
- if possible, use **local variables** instead
 - local variables are allocated on call and freed on return, automatically

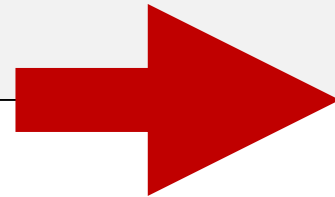
- Engineer for memory management

- define rules for which procedure is responsible for deallocation
- use explicit reference counting if multiple potential deallocators
- define rules for which pointers can be stored in data structures
- use coding conventions and documentation to ensure rules are followed

Strategy: Co-locating Allocation and Deallocation

- If a procedure returns the value of a dynamically allocated object
 - allocate that object in the **caller** and pass a pointer to it to the **callee**
 - good if caller does both malloc/free itself

```
struct MBuf* receive () {  
    struct MBuf* mBuf = malloc (sizeof (*mBuf));  
    ... // does something  
    return mBuf;  
}  
  
void foo () {  
    struct MBuf* mb = receive ();  
    free (mb);  
}
```

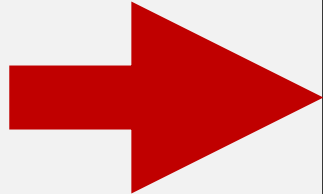


```
void receive (struct MBuf* mBuf) {  
    ... // does something  
}  
  
void foo () {  
    struct MBuf* mb = malloc (sizeof (*mBuf));  
    receive (mb);  
    free (mb);  
}
```

Strategy: Using Local Variables

- If a procedure does malloc and free
 - use a local variable instead of malloc
 - local variables are allocated on call and deallocated on return

```
struct MBuf* receive () {  
    struct MBuf* mBuf = malloc (sizeof (*mBuf));  
    ... // does something  
    return mBuf;  
}  
void foo () {  
    struct MBuf* mb = receive ();  
    free (mb);  
}
```



```
void receive (struct MBuf* mBuf) {  
    ... // does something  
}  
void foo () {  
    struct MBuf mb;  
    receive (&mb);  
}
```

Recap: Strings

- **A string “like this” is in C**
 - an array of characters
 - the end of the string is indicated by the first 0 (null) in the array
 - so, every string has a
 - maximum length (length of array – 1), a.k.a. size
 - length of the string, determined by the position of the first null
- **The standard C library has many operations on strings**
 - e.g. `strlen(s)` returns the length of a string

Example: String Copy (Version 1)

What can be improved in the following code?

```
char *copy(char *s) {  
    int len = strlen(s);  
    char *d = malloc(len + 1);  
    for (int i = 0; i <= len; i++)  
        d[i] = s[i];  
    return d;  
}
```

```
void foo(char *s) {  
    char *d = copy(s);  
    printf("%s\n", d);  
    free(d);  
}  
  
void bar() {  
    foo("Hello, World!");  
}
```

C library equivalent: strdup

Example: String Copy (Version 2)

Pros? Cons?

```
void copy(char *d, char *s) {  
    int len = strlen(s);  
    for (int i = 0; i <= len; i++)  
        d[i] = s[i];  
}
```

```
void foo(char *s) {  
    int len = strlen(s);  
    char *d = malloc(len+1);  
    copy(d, s);  
    printf("%s\n", d);  
    free(d);  
}
```

C library equivalent: strcpy

Example: String Copy (Version 3)

Pros? Cons?

```
void copy(char *d, int size,
          char *s) {
    int len = strlen(s);
    if (len > size-1) len = size-1;
    for (int i = 0; i < len; i++)
        d[i] = s[i];
    d[len] = 0;
}
```

```
void foo(char *s) {
    int len = strlen(s);
    char *d = malloc(len+1);
    copy(d, len+1, s);
    printf("%s\n", d);
    free(d);
}
```

C library equivalent: strncpy

These are ongoing concerns...

macOS Monterey 12.6 Security Update - **Sep 12, 2022**

Kernel

Available for: macOS Monterey

Impact: An application may be able to **execute arbitrary code with kernel privileges**. Apple is aware of a report that this issue may have been actively exploited.

Description: The issue was addressed with **improved bounds checks**.

CVE-2022-32917: an anonymous researcher

<https://support.apple.com/en-ca/HT213444>

Detecting Memory Problems: Valgrind

- Valgrind is a program that performs dynamic analysis of the runtime of a program
 - Example: `valgrind ./my_program`
- It runs your program and monitors dynamic allocation and deallocation
- It can tell if your program has:
 - memory leaks
 - use after free (dangling pointers)

Sometimes Things Aren't That Easy...

- Sometimes a reference needs to be passed among **multiple modules**
 - then each module knows when it needs and doesn't need reference
 - but these modules may not communicate that information between them
- What do you need to know in order to correctly **free** an object?
 - **free** should only happen if the object is **not in use**
 - you need to know if the object is still in use
 - you need to **keep track** of all its uses
 - when it starts being used
 - when it stops being used



https://www.reddit.com/r/saltierthancrait/comments/oiy94/impossible_adj_not_possible_unable_to_be_exist/

Example – problem with explicit free

What could go wrong here?

```
struct MBuf* aMB;

struct MBuf * receive () {
    struct MBuf* mBuf = malloc(sizeof(*mBuf));
    ...
    return mBuf;
}

void bar (struct MBuf* mb) {
    aMB = mb;
}

void foo () {
    struct MBuf* mb = receive ();
    bar(mb);
    free(mb);
}

void bat () {
    aMB->x = 0;
}
```

Example – interaction between foo and bar

```
struct MBuf* aMB = 0;
```

```
void bar (struct MBuf* mb) {  
    if (something_complicated_and_or_private)  
        aMB = mb;  
}
```

```
void foo () {  
    struct MBuf* mb = receive ();  
    bar (mb);  
    free (mb);    ??? if it's okay  
}
```

Reference Counting

- You can use **reference counting** to track object use
 - initialize the reference count to **1** (the caller has a reference)
 - any procedure that stores a reference (**starts** using object) **increments** the count
 - any procedure that discards a reference (**stops** using object) **decrements** the count
 - never call free directly
 - the object is **freed** when count goes to **zero**

Example – updating the code

```
struct MBuf* aMB = 0;
```

```
struct MBuf * receive () {  
    struct MBuf* mBuf = rc_malloc(sizeof(struct MBuf));  
    ...  
    return mBuf;  
}
```

```
void bar (struct MBuf* mb) {  
    // free existing reference in aMB  
    if (aMB != 0)  
        rc_free_ref (aMB);  
    rc_keep_ref (mb);  
    aMB = mb;  
}
```

```
void foo () {  
    struct MBuf* mb = receive ();  
    bar (mb);  
    rc_free_ref (mb);  
}
```

Reference Counting: An Implementation Approach

- Reference counter can be part of a **struct**
 - Updates to reference counter are done through special methods

```
struct buffer *rc_malloc() {  
    struct buffer *buf = malloc(sizeof(struct buffer));  
    buf->ref_count = 1;  
    return buf;  
}  
  
void rc_keep_ref(struct buffer *buf) {  
    buf->ref_count++;  
}  
  
void rc_free_ref(struct buffer *buf) {  
    buf->ref_count--;  
    if (buf->ref_count == 0)  
        free(buf);  
}
```

Reference Counting: Another Approach

```
void* rc_malloc (int n) {  
    int *ref_count = malloc (n + 8);  
    *ref_count = 1;  
    return ((void*) ref_count) + 8;  
}
```

alternative malloc with room for ref_count

```
void rc_keep_ref (void* p) {  
    int *ref_count = p - 8;  
    (*ref_count) ++;  
}
```

call when **reference added**

```
void rc_free_ref (void* p) {  
    int *ref_count = p - 8;  
    (*ref_count) --;  
    if (*ref_count == 0)  
        free (ref_count);  
}
```

call when **reference removed**

Reference Counting and Procedure Calls: Examples

- **When a reference is a parameter**

- the caller has a reference and maintains it for the duration of the call
- so, the callee need not call `keep_ref` **UNLESS**
 - callee saves the pointer someplace that will outlast the call (e.g., global variable)
 - callee returns the pointer to the caller

- **When a reference is a return value**

- the callee must have a reference to the value
- it passes that reference to the caller
 - the callee implicitly gives up its reference as part of the return
 - the reference is transferred to the caller
- the caller must call `free_ref` when it no longer stores the reference

- **When a reference is stored in a local variable**

- that variable goes away implicitly when the procedure returns
- and so the procedure must call `free_ref` before it returns

Be careful with references...

Consider this code:

```
int main(int argc, char **argv) {
    char *name = strdup("Quiz X");
    quiz_t quizzes[4];
    for (int i=0; i<4; i++) {
        name[5] = i + 49;
        quizzes[i] = quiz_create(name);
    }
    for (int i=0; i<4; i++) {
        printf("%d %s\n", i, quiz_get_name(quizzes[i]));
    }
}

quiz_t quiz_create(char* name) {
    struct quiz *quiz = malloc(sizeof(*quiz));
    quiz->name = name;
    return quiz;
}
```

it prints

```
0 Quiz 4
1 Quiz 4
2 Quiz 4
3 Quiz 4
```

why?

Nesting and Reference Counting

Now, how does this impact reference counting?

```
quiz_t quiz_create(char* name) {  
    struct quiz *quiz = malloc(sizeof(*quiz));  
    quiz->name = strdup(name);  
    return quiz;  
}  
  
void quiz_delete(quiz_t quiz) {  
    free(quiz->name);  
    free(quiz);  
}
```

What must we change in order to use reference counting?

Nesting and Reference Counting

What if there is more to the quiz structure?

```
struct quiz {
    char* name;
    int* grades;
    int num_grades;
};

quiz_t quiz_create(char* name) {
    quiz_t quiz = rc_malloc(sizeof(*quiz));
    int name_size = strlen(name) + 1;
    quiz->name = rc_malloc(name_size);
    strcpy(quiz->name, name, name_size);
    quiz->grades = NULL;
    return quiz;
}

void addGrades(quiz_t quiz, int *grades, int numGrades) {
    quiz->grades = rc_malloc(sizeof(*quiz->grades) * numGrades);
    for (int i = 0; i < numGrades; i++)
        quiz->grades[i] = grades[i];
}

void quiz_delete(quiz_t quiz) {
    rc_free_ref(quiz->name);
    rc_free_ref(quiz);
}
```

How to fix quiz_delete?

Deallocation in Java: Garbage Collection

- In Java objects are deallocated implicitly
 - the program never calls free
 - the runtime system tracks **every object reference**
 - a garbage collector runs **periodically** to deallocate unreachable objects
- Advantage compared to explicit free
 - no **dangling pointers**
 - (almost) no **memory leaks**
 - handles **reference cycles**
 - e.g., object a has pointer to b, object b has pointer to a
 - these would cause memory leak when using reference counters alone

Discussion

- What are advantages of explicit free?
- What are advantages of reference counting?
- What are advantages of garbage collection?
- Should we ignore deallocation in Java programs?

Memory Management in Java

- Memory leaks can still occur
 - when garbage collector fails to reclaim **unneeded** objects
 - it's a significant problem for long-running programs where garbage accumulates
- Why would an object not be reclaimed?
 - Java only reclaims **unreachable** objects
 - Unneeded/unused objects that keep references are not reclaimed
 - Collections and maps may maintain old unneeded objects

Aside: Avoid Unintended Retention

(not needed for CPSC 213)

- Solution: plan for memory deallocation explicitly
 - Remove references to unneeded objects
 - <http://www.ibm.com/developerworks/library/j-leaks/>
- Imperative approach: explicit reference annulling
 - explicitly set references to NULL when no longer needed
 - add close() or free() methods and call them explicitly
 - use try-finally block to ensure clean-up steps are always taken

Aside: Reference Objects

(not needed for CPSC 213)

- Declarative approach: reference objects
 - refer to objects without requiring long-term retention
 - store object references that garbage collector can reclaim
- ```
WeakReference<Widget> weakRef = new WeakReference<Widget>(widget);
Widget widget = weakRef.get();
```
- different levels of reference “stickiness”
    - soft: discarded only when new allocations put pressure on available memory
    - weak: discarded on next GC cycle when no stronger reference exists
    - phantom: unretrievable, used to register with GC reference queue
  - referenced object can be reclaimed, but WeakReference not
    - Alternative: WeakHashMap